

Assignment 3 IIR filters: Contactless Light Switch

1.1 Introduction

Detecting small physical changes in real time is a common challenge in sensor applications, especially when the measured signal is extremely weak and easily corrupted by noise. In this project, we built a contactless motion controlled LED system using the CDM324 microwave Doppler radar sensor. The raw output of the CDM324 is centred around 0 V and has a very small amplitude, far below the resolution of the Arduino Nano's 10-bit analogue to digital converter. Because the Arduino Nano can only measure voltages between 0 V and 5 V, the sensor output must first be shifted using a DC offset so that the entire waveform lies within the measurable voltage range.

Even after applying a DC offset, the radar output remains too small to be detected reliably. Therefore, the signal was amplified by a factor of approximately 50 using an analogue opamp stage. This amplification increases the visibility of the Doppler motion signal but also amplifies unwanted noise. As a result, real time digital filtering is required to remove the DC component, suppress high frequency noise, and extract the meaningful motion related variations from the amplified radar signal.

The project uses an Arduino Nano as the data acquisition device, running at a sampling rate of 1 kHz through pyFirmata2's interface. The filtered signal is processed in Python using a custom chain of second order IIR filter sections implemented. A bandpass response is constructed by cascaded highpass and lowpass Butterworth sections, designed specifically to clean the radar output in real time. After filtering, the remaining clean Doppler envelope is used to detect motion and toggle an LED without physical contact.

The aim of this work is to demonstrate:

1. a real time sensing problem that requires IIR filtering,
2. the design and implementation of second order filter sections and cascaded IIR structures,
3. real time performance verification using sampling rate monitoring,
4. and successful physical behaviour of the system through filtered signal analysis and a real time demonstration.

Overall, this project shows how analogue conditioning and digital filtering can be combined to extract meaningful information from a noisy microwave radar sensor and use it to control an embedded system in real time.

2.1 Sensor Module

The main component used in this project was a CDM324 24 GHz Doppler Radar. Pictured below is a schematic of the radar, as well as an image of the module.

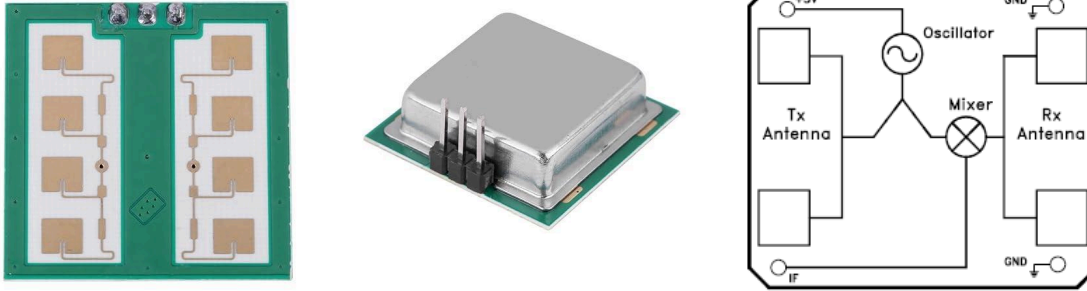


Figure: (Left) An image of the radiating surface of the radar module. (Center) An image of the back of the radar. (Right) A simplified version of the radar schematic

The radar itself had three pins, +5V, GND, and IF. The +5V input is used to power an oscillator that produces a 24 GHz signal. This signal is used to excite one half of the antenna array (Tx), which transmits the signal towards the target. The other half of the array (Rx) receives the signal, which is mixed with the 24 GHz signal. The mixer takes in these two sine waves and produces sine waves with frequencies $m\omega_1 + n\omega_2$ for $m, n = \pm 1, 2, 3, \dots$. This is then passed through an LPF in the radar (not shown here) which isolates $\omega_1 - \omega_2$, and gives us a sine wave with a frequency equal to the difference in frequencies between the transmitted and received signals. Because of the Doppler effect, a wave reflecting off a target moving with velocity v will have a slightly different frequency to the incident wave. This frequency difference allows us to work out the velocity of the target, given by:

$$v = \frac{2f_c}{f_d c}$$

Where f_c is the carrier frequency (24 GHz), c is the speed of light, and f_d is mixer output frequency. f_d is the frequency of the wave that is output from the mixer, and which serves as our signal in this project.

2.2 Circuit Diagram

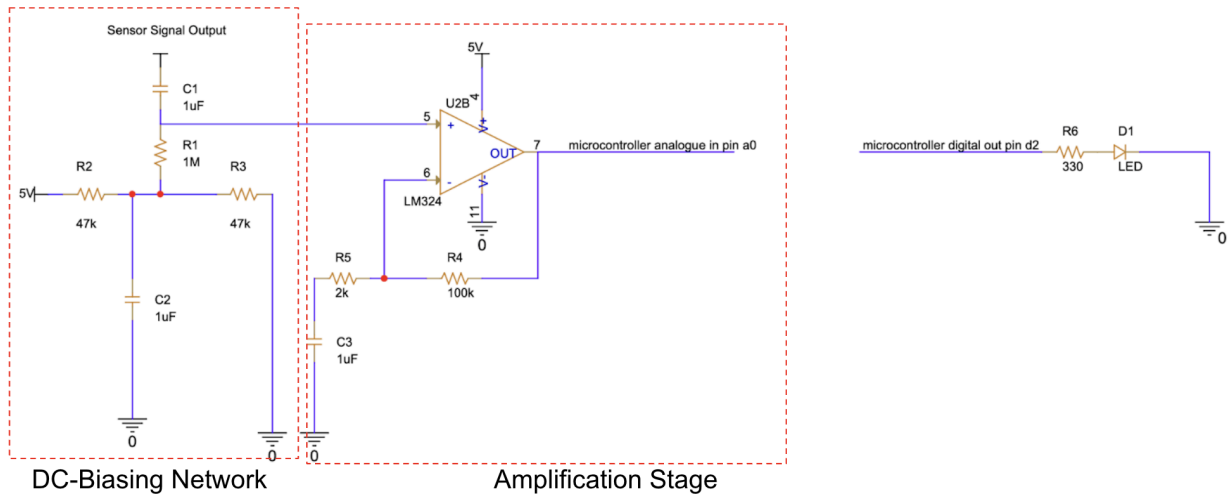


Figure: The schematic of the circuit used to DC-bias and then amplify the radar signal

Shown above is a schematic of the circuit used to process the radar signal before sampling. This is necessary for two reasons:

1. The raw radar output signal is extremely small without any amplification, and is on the order of ~ 50 mV even for significant motion. The Arduino ADC's minimum detectable signal is 4.88 mV.
2. The raw radar output signal is centered at 0V, but the Arduino only accepts inputs between 0 - 5V.

In order to solve these problems, we designed the analog circuit above. In the DC-Biasing network, we used a $1\ \mu\text{F}$ capacitor to decouple the DC component of the signal and from the AC component, which was then shifted up by ~ 1.5 V using a voltage divider network. This signal was fed into an op-amp.

The amplification stage uses a non-inverting amplifier. The AC voltage gain of the circuit is ~ 50 , and the DC voltage gain of the circuit is 1 due to the presence of the $1\ \mu\text{F}$ capacitor to ground. The output of the op-amp was then fed into the Arduino ADC.

To trigger the LED light, we used a digital signal from the Arduino's D2 pin.

2.3 Photos of the Setup

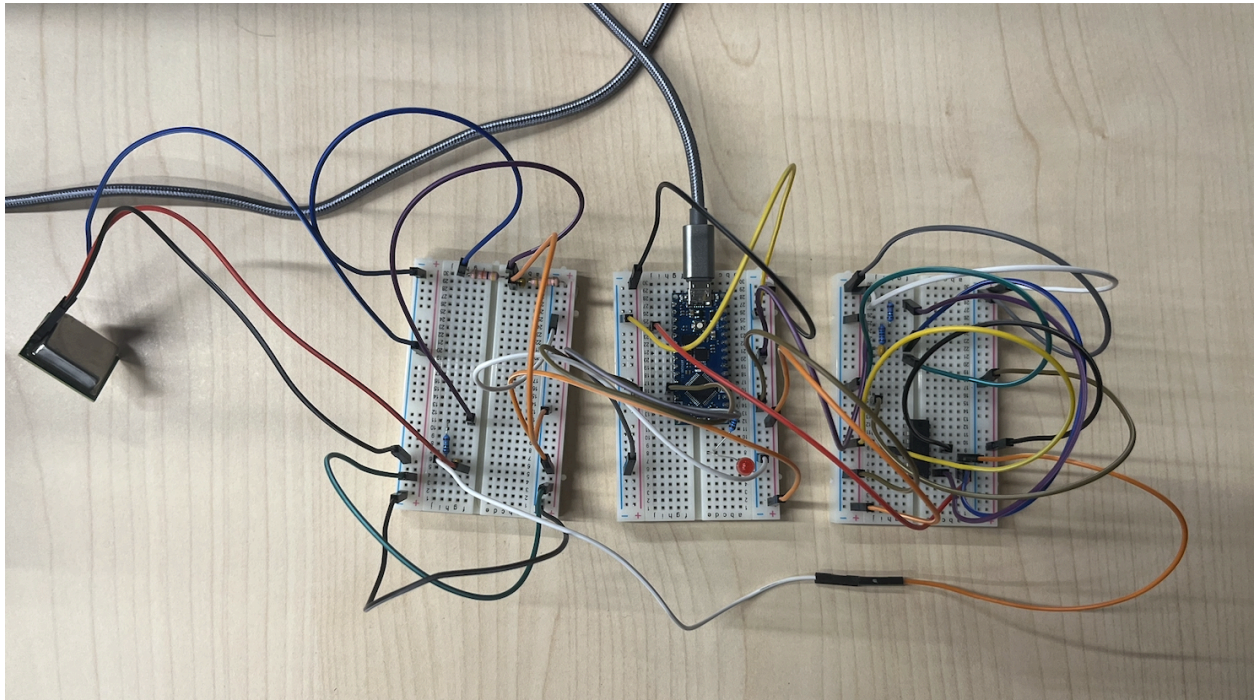


Figure: An image of our circuit. The leftmost breadboard contains the DC-biasing network. The central breadboard contains the Arduino and LED. The rightmost breadboard contains the amplification stage.

Shown above is our breadboard implementation of the circuit. The length of wires and the use of breadboards created a lot of additional noise in our signal.

3.1 Dataflow Diagram

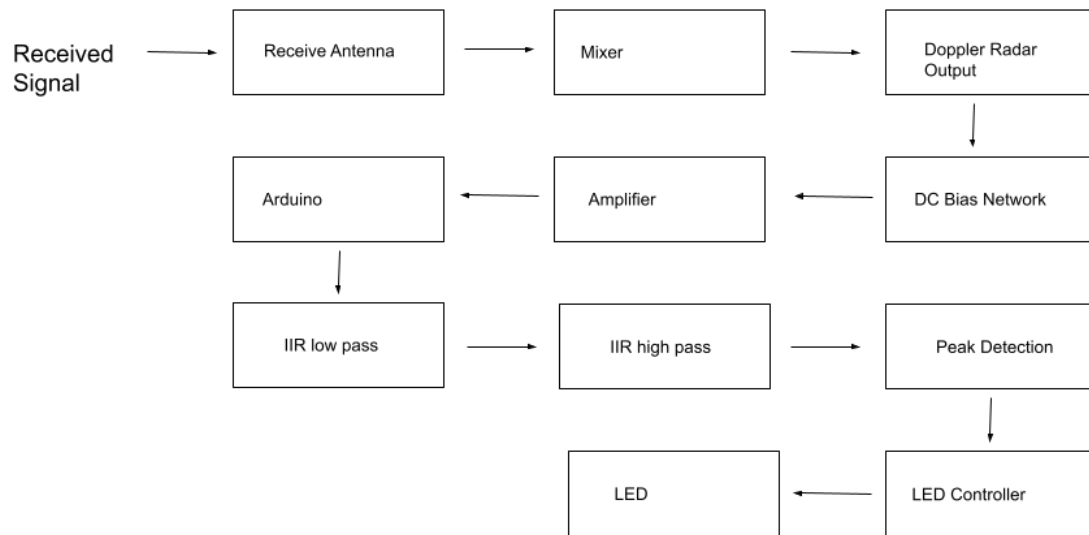


Figure: Dataflow diagram of our project

4.1 Signal Characteristics and Why Filtering is Needed

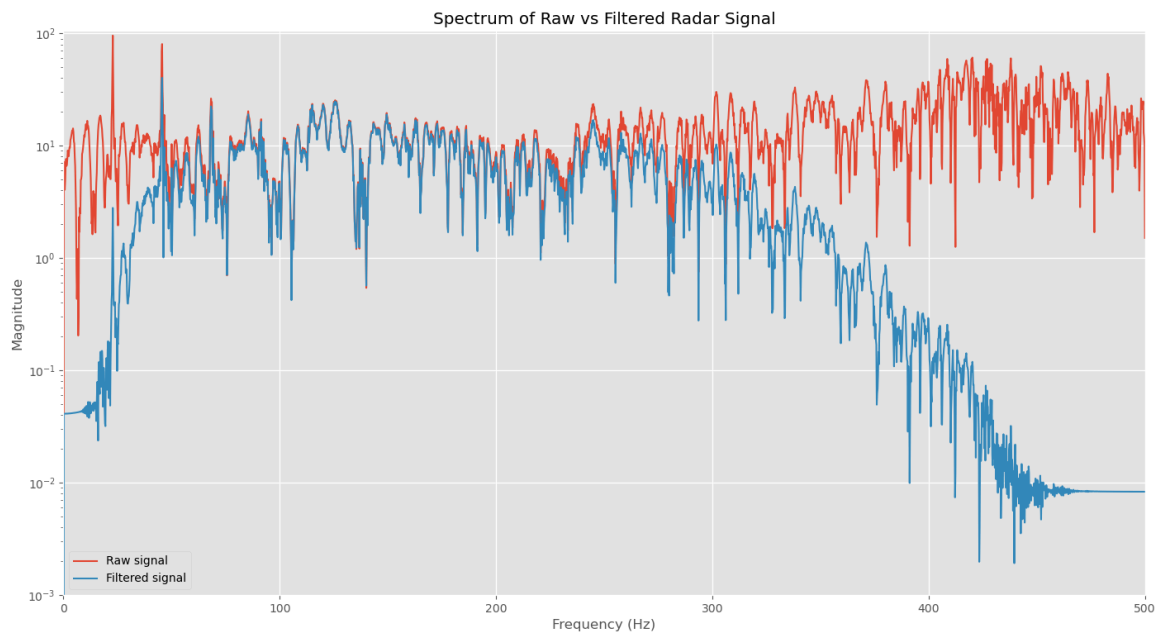


Figure: Frequency spectrum of the raw (red) and filtered (blue) radar signals

The above figure shows the magnitude spectrum of the raw and filtered radar signals, taken when the radar is pointing at a stationary target. In the raw spectrum, there is a strong concentration of energy at very low frequencies, especially below about 40 Hz. This is caused by the DC offset, slow drift in the analogue circuitry and low frequency environmental noise such as vibrations. For motion detection this part of the spectrum is not useful, because it does not

correspond to actual hand movement but to static or slowly varying offsets. Therefore, the highpass part of the IIR bandpass filter is chosen to remove frequencies below 40 Hz, which effectively suppresses this low frequency noise and drift. From the Doppler equation, a frequency of 40 Hz corresponds to 0.9 km/h, which is well below the average walking speed of a human, and is much slower than any meaningful hand motion.

At the other end of the spectrum, frequencies above roughly 300 Hz are also not relevant for our application. For a 24 GHz CDM324 radar, the Doppler frequency produced by typical hand motions is in the order of a few tens to a few hundred hertz, depending on the speed of the movement. Even very fast hand motions do not generate Doppler components anywhere near 300 Hz. The energy seen above this range in the raw spectrum is therefore mainly high frequency noise from the sensor and the analogue amplifier. For this reason, the lowpass part of the IIR bandpass filter is set to 300 Hz, which removes these meaningless high frequency components while preserving the Doppler band where the actual motion information resides. This explains why the filtered spectrum in the above figure is confined to the 40-300 Hz band.

5.1 Filter requirements and overall approach

The goal of the digital filter is to extract the Doppler motion signal from the noisy, amplified radar output. As discussed earlier, useful motion information lies roughly between 40 Hz and 300 Hz, while frequencies below 40 Hz are dominated by DC offset and drift, and frequencies above 300 Hz are mainly high frequency noise. The system runs at a sampling rate of 1000 Hz. To meet these requirements we designed a bandpass IIR filter built from second order Butterworth sections. A highpass section with a cutoff of 40 Hz is cascaded with a lowpass section with a cutoff of 300 Hz. In our implementation, we used three cascaded 2nd order lowpass filters, followed by three cascaded 2nd order high-pass filters, to ensure a steep rolloff at the cutoff frequencies.

5.2 Coefficient generation (coefficient_generation.py)

This file generates the numerical coefficients for a second order Butterworth lowpass or highpass filter. The transfer functions, H_{LPF} and H_{HPF} , for low-pass and high pass 2nd order Butterworth filters are shown below:

$$H_{LPF}(s) = \frac{1}{s^2 + \sqrt{2}s + 1}$$

$$H_{HPF}(s) = \frac{s^2}{s^2 + \sqrt{2}s + 1}$$

In order to convert these transfer functions into IIR filter coefficients the following steps were taken:

- Normalise the cutoff frequency: The cutoff frequency is first converted into a digital angular frequency relative to the sampling rate.
- Prewarp the frequency: Because digital filters are designed through the bilinear transform, the cutoff is prewarped using a tangent operation so that the digital filter matches the intended analogue cutoff.
- Compute the core Butterworth terms: Using the prewarped value, the code computes three denominator terms (A_0 , A_1 , A_2) that define the pole locations of a standard 2nd order filter.
- Form the numerator (low pass filter or high pass filter): For the lowpass filter, the numerator is symmetric so that low frequencies pass and high frequencies are attenuated. For the highpass filter, the numerator is flipped to introduce a zero at DC, removing slow drift and low frequency components.
- Normalise all coefficients: All coefficients are divided by A_0 so the resulting values can be used directly in the difference equation.
- Return the coefficients: Each function outputs the five coefficients (a_1 , a_2 , b_0 , b_1 , b_2) that define a single second order IIR filter section.

These coefficients are then passed to the IIR2ndOrder class, which performs the actual real time filtering sample by sample.

5.3 Second order IIR filter ([IIR2ndOrder.py](#))

This file implements a single second order IIR filter that operates on one sample at a time. This IIR2ndOrder class takes the five filter coefficients (a_1 , a_2 , b_0 , b_1 , b_2) in its constructor, stores two short internal buffers x and y to keep the last three input and output samples.

The method `dofilter(v)` is called for every new input sample to:

- Shift the buffers: The previous input and output samples are moved one position down in the x and y arrays, so that $x[1]$ and $x[2]$ always contain the two past inputs and $y[1]$ and $y[2]$ contain the two past outputs.
- Insert the new sample: The current input v is stored in $x[0]$.
- Compute the new output: A weighted sum of the current and past inputs, minus a weighted sum of the past outputs, is calculated using the stored coefficients. This produces the current output sample $y[0]$.
- Return the filtered value: The method returns $y[0]$, which is the filtered version of the input sample.

This class provides a simple, constant time implementation of a second order IIR section that can be reused multiple times in a filter chain to build higher order responses.

5.4 Cascaded filter chain ([IIRChain.py](#))

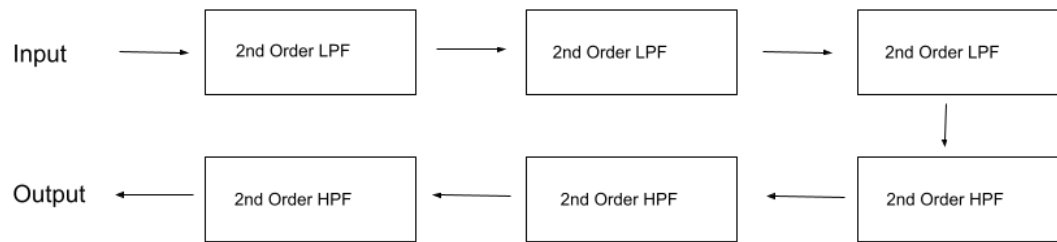


Figure: A diagram showing the cascaded filter chain

The file `IIRChain.py` defines a simple structure for combining multiple second order IIR filter sections into a single, higher order filter. The class keeps a list of individual `IIR2ndOrder` filters, and new sections can be added using the `addFilter()` method. When a new input sample arrives, the `dofilter(x)` function forwards the sample through each section in sequence. The output of one filter becomes the input to the next, and the final value is returned. This chaining mechanism allows several second order filters to work together, producing a steeper and more selective overall response than a single section could achieve on its own.

5.5 Radar Signal Processing ([Radar.py](#))

The file `Radar.py` defines the main digital signal processing component used in the project. The `RadarDSP` class is responsible for building the bandpass filter, applying it to each incoming sample, and storing the raw and filtered data for real time plotting.

When the class is initialised, it receives the sampling rate and the cutoff frequencies for the highpass and lowpass parts of the band-pass filter. It then calls `_build_filter_chain()`, which uses the coefficient generation functions to create several identical lowpass and highpass second order sections. These sections are added to an `IIRChain` instance, forming a steeper bandpass response by cascading multiple simple filters.

The method `process_sample(voltage)` is called for every new analogue reading from the radar sensor. It increments an internal sample counter, sends the sample through the entire filter chain, and returns the filtered output. At the same time, the method stores the timestamp, raw input value, and filtered value in fixed size buffers (Python deque objects). These buffers are later used by the plotting code to display the real time raw and filtered signals. This class therefore acts as the central DSP unit of the system, handling both filtering and data collection in a structure that supports real time operation.

5.6 Motion detection and LED control ([LED.py](#))

The file `LED.py` contains the logic that converts the filtered radar signal into a simple on/off control for the LED. The `MotionLedController` class receives the filtered signal and applies a threshold based decision rule to determine when motion has occurred.

The controller keeps track of the current motion state and stores the last sample index at which the LED was toggled. When a new filtered value arrives, the `update()` method compares it to a high threshold. If the signal exceeds this threshold and enough samples have passed since the last toggle, the LED state is inverted and motion is marked as active. Once active, the controller waits until the filtered value drops below a lower threshold before returning to the idle state.

This two threshold scheme introduces hysteresis, which prevents rapid flickering due to noise. The class also provides a `turn_off()` method that resets the LED and internal state. Overall, this module translates the cleaned signal into stable LED behaviour, enabling the contactless on/off demonstration.

5.7 Real time acquisition and application control ([main.py](#))

The file `main.py` contains the real time loop that connects all parts of the system together. It initialises the Arduino via `pyFirmata2`, sets the sampling rate to 1 kHz, and registers a callback function that is automatically triggered every time a new analogue sample is received from the radar sensor.

Inside this callback, the raw voltage reading is scaled to 0–5 V and passed into the `RadarDSP` object, which applies the cascaded IIR bandpass filter. The filtered value is then forwarded to the `MotionLedController`, which updates the LED state based on the motion detection logic. The callback also tracks the number of processed samples and periodically computes the measured sampling rate, allowing us to verify that the system maintains real time performance.

For visualisation, the script creates two live matplotlib plots that display the raw and filtered signals over a sliding time window. The plotting buffers are filled inside `RadarDSP`, and the animation function simply refreshes the curves at regular intervals without interrupting the real time data processing. When the window is closed, the script safely shuts down the LED and closes the Arduino connection.

Overall, `main.py` acts as the integration layer of the project: it manages data acquisition, real time DSP, motion detection, and sampling rate monitoring.

5.8 IIR Filter Unit Tests ([test_IIR.py](#))

The file `test_IIR.py` contains a small set of unit tests that verify the correctness of both the `IIR2ndOrder` class and the cascaded filter structure implemented in `IIRChain`.

For the second order section, two simple test cases are used. The first test constructs an identity filter, where the output should match the input exactly; this confirms that the difference equation implementation and the internal sample shifting logic are correct. The second test uses coefficients that turn the filter into a three point moving sum. By applying an impulse input and checking the expected output sequence, the test confirms that the filter correctly handles past inputs.

The chain behaviour is validated by connecting two identical second order sections in series. When an impulse is passed through this chain, the expected output is the convolution of the individual responses. The test checks that the computed output matches this known result, demonstrating that the chaining mechanism in IIRChain works properly.

Together, these show that both individual filter sections and cascaded configurations behave as expected before they are used in the real time signal processing.

6.1 Sampling Rate Verification

The system is designed to run at a sampling rate of 1 kHz using the Arduino and pyFirmata2. However, in a real time setup the effective sampling rate can deviate from the requested value due to USB delays, operating system scheduling and Python overhead. To monitor this, we added code in main.py that continuously measures the actual sampling rate while the application is running. In the callback `a0_callback`, every incoming sample increments the internal sample counter `dsp.current_sample`. At the same time, we keep track of the time using `time.time()`. Once per second (`FS_REPORT_INTERVAL_S = 1.0`), the code calculates how many samples have been processed since the last report and divides this by the elapsed time. The system then prints the calculated actual sampling rate in Hz alongside the expected sampling rate. The printed values are very close to 1kHz which confirms that the system is operating in real time and that the filter is running at the intended rate.

Checking the sampling rate is important for two reasons. First, the IIR band-pass filter was designed for a specific sampling rate. If the real rate changes, the cutoff frequencies shift and the filter may no longer focus on the part of the signal that contains the motion information. Second, if the sampling rate becomes unstable or drops too low, the real-time behaviour of the application is affected, which can lead to delays or missed motion events. Continuously monitoring the sampling rate ensures that both the filter and the overall system operate as intended.

7.1 Real Time Results

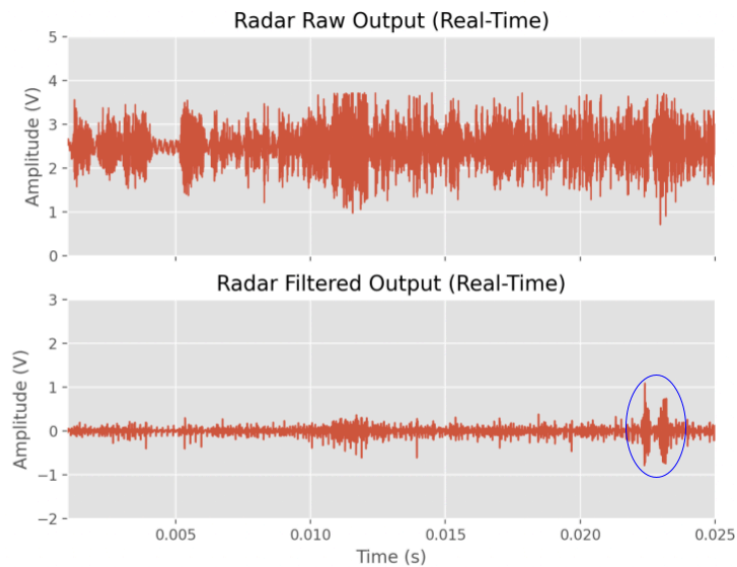


Figure: Comparison of raw radar output signal and filtered radar output signals

The above figure shows a comparison of the radar's output signal with and without filtering. In the unfiltered plot, there is significant noise, which makes it impossible to detect motion. In the filtered plot, there is a clear detection present (circled in blue) that would have been masked by the noise if it wasn't for the IIR filter. When the motion is detected the LED light goes on and if it detects another motion it goes off. If it doesn't detect any other motion it either remains on or off depending on the state.

8.1 Youtube Demo Link

<https://youtu.be/JUmlmh1xke4>

9.1 Critical Analysis

Overall, we were successful in detecting hand motion and using it to toggle an LED light. This was achieved by amplifying our signal and then using a cascaded IIR chain to filter out DC and noise while preserving our signal. To check for motion, we looked for signals that exceeded a certain voltage threshold, allowing us to prevent spurious light toggling, and identify real movement.

However, we faced many limitations. Firstly, our amplifier circuit introduced a lot of noise into our signal. While some of this was able to be filtered out, there was still significant noise present in the passband of our filter. This meant that we had to try to strike a balance between rejecting noise while still being able to detect normal hand movements. Although we spent a lot of time testing and tuning our filter passband, with more time we might have been able to improve our IIR filter. Another significant source of noise was poor quality of connections between wires and

the breadboard. These loose connections meant that our circuit was very sensitive, and was also a source of noise. In order to fix this, we could have soldered connections together, and if we had more time, designed a PCB to house our circuitry and allow us to carefully manage our analog signal. Perhaps most significantly, the radar module we chose did not produce a strong enough signal to enable a greater range of detection. Initially we had hoped to be able to detect people entering and leaving rooms, and even to calculate their velocity in real time. This was based on the CDM324's advertised range of 15m. However, the radar's signal was very small, and when combined with our breadboard amplifier circuit, produced too much noise across all frequencies for us to have good resolution.

APPENDIX

coefficient_generation.py :

```
import numpy as np

def butter_lpf_2nd(cutoff, fs):

    alpha = np.sqrt(2)

    omega_c = (2*np.pi*cutoff)/fs

    omega_ac = np.tan(omega_c / 2)

    K = 1 / omega_ac

    A0 = K**2 + alpha*K + 1
    A1 = -2*(K**2) + 2
    A2 = K**2 - alpha*K + 1

    b0 = 1/A0
    b1 = 2/A0
    b2 = 1/A0

    a1 = A1/A0
    a2 = A2/A0

    return a1,a2,b0,b1,b2

def butter_hpf_2nd(cutoff, fs):

    alpha = np.sqrt(2)

    omega_c = (2*np.pi*cutoff)/fs # digital cutoff
```

```

omega_ac = np.tan(omega_c / 2) # prewarped analog cutoff

K = 1 / omega_ac

A0 = K**2 + alpha*K + 1
A1 = -2*(K**2) + 2
A2 = K**2 - alpha*K + 1

# High-pass numerator
b0 = (K**2) / A0
b1 = -2*(K**2) / A0
b2 = (K**2) / A0

# Denominator (same poles as LPF)
a1 = A1 / A0
a2 = A2 / A0

return a1, a2, b0, b1, b2

```

IIR2ndOrder.py :

```

import numpy as np

class IIR2ndOrder:

    def __init__(self, a1, a2, b0, b1, b2):

        self.a1 = a1
        self.a2 = a2
        self.b0 = b0
        self.b1 = b1
        self.b2 = b2
        self.x = [0, 0, 0]
        self.y = [0, 0, 0]

    def dofilter(self, v):

        #shifting values
        self.y[2] = self.y[1]

```

```

        self.y[1] = self.y[0]
        self.x[2] = self.x[1]
        self.x[1] = self.x[0]

        #performing output calculation

        self.x[0] = v

        self.y[0] = self.b0 * self.x[0] + self.b1 * self.x[1] + self.b2 * self.x[2] -
self.a1 * self.y[1] - self.a2 * self.y[2]

        return self.y[0]

```

IIRChain.py :

```

import numpy as np
from IIR2ndOrder import IIR2ndOrder
class IIRChain:

    def __init__(self):

        self.filterList = []

    def addFilter(self, f : IIR2ndOrder):

        self.filterList.append(f)

    def dofilter(self, x):

        y = x
        for filter in self.filterList:

            y = filter.dofilter(y)

        return y

```

LED.py :

```

class MotionLedController:

    def __init__(self, led_pin, th_high: float, th_low: float,
min_toggle_interval_samples: int = 0):

```

```

self.led = led_pin
self.th_high = th_high
self.th_low = th_low
self.min_toggle_interval = min_toggle_interval_samples

self.motion_state = 0 # 0 = IDLE, 1 = ACTIVE
self.led_state = 0
self.last_toggle_sample = 0

if self.led is not None:
    self.led.write(0)

def update(self, y_clean: float, sample_index: int) -> int:

    val = y_clean

    if self.motion_state == 0:
        # rising through threshold
        if val > self.th_high:
            if (sample_index - self.last_toggle_sample) >=
self.min_toggle_interval:
                self.motion_state = 1
                self.led_state = 1 - self.led_state
                if self.led is not None:
                    self.led.write(self.led_state)
                self.last_toggle_sample = sample_index
            else:
                # falling back below low threshold
                if val < self.th_low:
                    self.motion_state = 0

        return self.led_state

def turn_off(self):
    self.led_state = 0
    if self.led is not None:
        self.led.write(0)

```

Radar.py:

```
from collections import deque

from IIR2ndOrder import IIR2ndOrder
from IIRChain import IIRChain
from coefficient_generation import butter_hpf_2nd, butter_lpf_2nd

class RadarDSP:

    def __init__(self, fs: float, hpf_cutoff: float, lpf_cutoff: float, plot_window_s: float):

        self.FS = fs

        # Sample counter
        self.current_sample = 0

        # plotting buffers (raw and filtered signals)
        self.PLOT_WINDOW_S = plot_window_s
        max_points = int(self.PLOT_WINDOW_S * self.FS)

        self.times = deque([], maxlen=max_points)
        self.raw_values = deque([], maxlen=max_points)
        self.filtered_values = deque([], maxlen=max_points)

        self._build_filter_chain(hpf_cutoff, lpf_cutoff)

    def _build_filter_chain(self, hpf_cutoff: float, lpf_cutoff: float) -> None:

        # design 2nd order butterworth low pass and high pass filters sections
        lpf_a1, lpf_a2, lpf_b0, lpf_b1, lpf_b2 = butter_lpf_2nd(lpf_cutoff, self.FS)
        hpf_a1, hpf_a2, hpf_b0, hpf_b1, hpf_b2 = butter_hpf_2nd(hpf_cutoff, self.FS)

        # create multiple sections for a steeper filter
        lpf_1 = IIR2ndOrder(lpf_a1, lpf_a2, lpf_b0, lpf_b1, lpf_b2)
        lpf_2 = IIR2ndOrder(lpf_a1, lpf_a2, lpf_b0, lpf_b1, lpf_b2)
        lpf_3 = IIR2ndOrder(lpf_a1, lpf_a2, lpf_b0, lpf_b1, lpf_b2)

        hpf_1 = IIR2ndOrder(hpf_a1, hpf_a2, hpf_b0, hpf_b1, hpf_b2)
        hpf_2 = IIR2ndOrder(hpf_a1, hpf_a2, hpf_b0, hpf_b1, hpf_b2)
```



```

        hpf_3 = IIR2ndOrder(hpf_a1, hpf_a2, hpf_b0, hpf_b1, hpf_b2)

        self.filter_chain = IIRChain()
        for section in [lpf_1, lpf_2, lpf_3, hpf_1, hpf_2, hpf_3]:
            self.filter_chain.addFilter(section)

    def process_sample(self, voltage: float) -> float:

        self.current_sample += 1

        # apply the IIR bandpass filter
        y = self.filter_chain.dofilter(voltage)

        # update plotting buffers
        t = self.current_sample / self.FS
        self.times.append(t)
        self.raw_values.append(voltage)
        self.filtered_values.append(y)

        return y

```

main.py :

```

import pyfirmata2
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import time
import numpy as np

from Radar import RadarDSP
from LED import MotionLedController

#application parameters
FS = 1000 # sampling rate in hz
HPF_CUTOFF = 40 # high pass cutoff freq
LPF_CUTOFF = 300 # low pass cutoff freq

PLOT_WINDOW_S = 5.0 # seconds of data shown in plot

# envelope / motion detection for LED

```

```

ENV_ALPHA = 0.05 # envelope smoothing factor
TH_HIGH = 0.75 # motion start threshold
TH_LOW = 0.2 # motion end threshold
MIN_LED_TOGGLE_INTERVAL_SAMPLES = int(1.0 * FS) # e.g. 1 second at 1 kHz

FS_REPORT_INTERVAL_S = 1.0 #printing measured sampling rate every second

# aurdino setup
board = pyfirmata2.Arduino(pyfirmata2.Arduino.AUTODETECT)
board.samplingOn(1) # request 1 kHz sampling from pyFirmata2

# LED on digital pin D2
led_pin = board.get_pin('d:2:o')

# creating the radar and led controller objects, radar object handles IIR filter and
plotting buffers
dsp = RadarDSP(
    fs=FS,
    hpf_cutoff=HPF_CUTOFF,
    lpf_cutoff=LPF_CUTOFF,
    plot_window_s=PLOT_WINDOW_S
)

# led controller
motion_led = MotionLedController(
    led_pin=led_pin,
    #env_alpha=ENV_ALPHA,
    th_high=TH_HIGH,
    th_low=TH_LOW,
    min_toggle_interval_samples=MIN_LED_TOGGLE_INTERVAL_SAMPLES
)

# sampling rate monitor state
last_fs_report_time = time.time()
last_fs_report_sample = 0

def a0_callback(data):
    global last_fs_report_time, last_fs_report_sample

    if data is None:
        return

```

```

# firmata gives 0..1 , convert to 0..5V
voltage = data * 5.0

# process through cascaded IIR filters
y = dsp.process_sample(voltage)

# update LED logic based on filtered signal
motion_led.update(y, dsp.current_sample)

#checking sampling rate
now = time.time()
dt = now - last_fs_report_time

if dt >= FS_REPORT_INTERVAL_S:
    # how many samples since last report
    dsamples = dsp.current_sample - last_fs_report_sample

    if dt > 0:
        measured_fs = dsamples / dt
        print(f"Measured sampling rate: {measured_fs:.1f} Hz (expected {FS} Hz)")

    # reset for next interval
    last_fs_report_time = now
    last_fs_report_sample = dsp.current_sample

a0 = board.get_pin('a:0:i')
a0.register_callback(a0_callback)
a0.enable_reporting()

print("Running real time radar DSP at 1 kHz.")

#plotting
plt.style.use('ggplot')

fig, (ax_raw, ax_filt) = plt.subplots(2, 1, sharex=True, figsize=(8, 6))

line_raw, = ax_raw.plot([], [], lw=1.0)
ax_raw.set_title("Radar Raw Output (Real-Time)")
ax_raw.set_ylabel("Amplitude (V)")
ax_raw.set_ylim(0.5, 4.5)

```

```

# filtered signal line
line_filt, = ax_filt.plot([], [], lw=1.2)
ax_filt.set_title("Radar Filtered Output (Real-Time)")
ax_filt.set_xlabel("Time (s)")
ax_filt.set_ylabel("Amplitude (V)")
ax_filt.set_ylim(-2, 2)
ax_filt.set_xlim(0, PLOT_WINDOW_S)

def update_plot(frame):

    if len(dsp.times) > 1:
        ax_filt.set_xlim(dsp.times[0], dsp.times[-1])
        line_raw.set_data(dsp.times, dsp.raw_values)
        line_filt.set_data(dsp.times, dsp.filtered_values)
    return line_raw, line_filt

ani = animation.FuncAnimation(fig, update_plot, interval=150, blit=True)

try:
    plt.show()
except KeyboardInterrupt:
    pass

'''
if len(dsp.raw_values) > 0 and len(dsp.filtered_values) > 0:
    # convert deque to numpy arrays
    raw = np.array(dsp.raw_values, dtype=float)
    filt = np.array(dsp.filtered_values, dtype=float)

    # remove DC component for better visualisation of spectrum
    raw = raw - np.mean(raw)
    filt = filt - np.mean(filt)

    N = len(raw)
    # one-sided frequency axis (0 .. Fs/2)
    freqs = np.fft.rfftfreq(N, d=1.0 / FS)

    # FFT magnitude
    RawSpec = np.abs(np.fft.rfft(raw))
    FiltSpec = np.abs(np.fft.rfft(filt))

    plt.figure(figsize=(8, 4))

```

```

plt.semilogy(freqs, RawSpec, label="Raw signal")
plt.semilogy(freqs, FiltSpec, label="Filtered signal")

# frequency range
plt.xlim(0, 500)

# magnitude range (10^-3 to max)
plt.ylim(1e-3, max(RawSpec) * 1.1)

plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.title("Spectrum of Raw vs Filtered Radar Signal")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
'''

print("Exiting...")
motion_led.turn_off()
board.exit()

```

test_IIR.py :

```

import unittest

from IIR2ndOrder import IIR2ndOrder
from IIRChain import IIRChain

class TestIIR2ndOrder(unittest.TestCase):

    def test_identity_filter(self):
        # b0 = 1, b1 = b2 = 0, a1 = a2 = 0 -> y[n] = x[n] output should be the same
        # as the input

        filt = IIR2ndOrder(a1=0.0, a2=0.0, b0=1.0, b1=0.0, b2=0.0)

        x_seq = [0.0, 1.0, -0.5, 2.0, 0.0]
        for x, expected in zip(x_seq, x_seq):
            y = filt.dofilter(x)

```

```

        self.assertAlmostEqual(y, expected, places=7)

def test_moving_sum(self):
    #b0 = b1 = b2 = 1, a1 = a2 = 0 -> y[n] = x[n] + x[n-1] + x[n-2]
    #expected output [1, 1, 1, 0, 0, ...]
    filt = IIR2ndOrder(a1=0.0, a2=0.0, b0=1.0, b1=1.0, b2=1.0)

    # impulse input
    x_seq = [1.0, 0.0, 0.0, 0.0, 0.0]
    expected = [1.0, 1.0, 1.0, 0.0, 0.0]

    y_seq = []
    for x in x_seq:
        y_seq.append(filt.dofilter(x))

    for y, e in zip(y_seq, expected):
        self.assertAlmostEqual(y, e, places=7)

class TestIIRChain(unittest.TestCase):

    def test_filter_chain(self):

        f1 = IIR2ndOrder(a1=0.0, a2=0.0, b0=1.0, b1=1.0, b2=1.0)
        f2 = IIR2ndOrder(a1=0.0, a2=0.0, b0=1.0, b1=1.0, b2=1.0)

        chain = IIRChain()
        chain.addFilter(f1)
        chain.addFilter(f2)

        # impulse input
        x_seq = [1.0, 0.0, 0.0, 0.0, 0.0]
        # convolution of [1,1,1] with [1,1,1] -> [1,2,3,2,1]
        expected = [1.0, 2.0, 3.0, 2.0, 1.0]

        y_seq = []
        for x in x_seq:
            y_seq.append(chain.dofilter(x))

        for y, e in zip(y_seq, expected):
            self.assertAlmostEqual(y, e, places=7)

```

```
if __name__ == "__main__":  
    unittest.main()
```